Notes on Ch24-Web Scraping

Norman Lim

2025-08-15

Main Idea: Introduce the basics of web scraping using **rvest**.

Prerequisites

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   3.5.2     v tibble    3.3.0
## v lubridate 1.9.4     v tidyr     1.3.1
## v purrr     1.0.4
## -- Conflicts ---------------------------------------- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(rvest)
```

```
##
## Attaching package: 'rvest'
##
## The following object is masked from 'package:readr':
##
##     guess_encoding
```

Scraping ethics and legalities

General principles:

- Legalities depend a lot on where you live.
- If data is public, factual, and non-personal, it is likely ok.
- If data is not public, non-personal, or not factual, talk to a lawyer first.
- If scraping many pages, make sure to wait a little more between each request.

- If available, check the terms and service of the website.
- Be careful when scraping personally identifiable information like names, email addresses, phone numbers, dates of birth, etc.
- Look at copyright laws and see what's protected. They generally apply to original works of authorship, but scraping "facts" are usually excluded, but this may vary depending on location/country.
- One may scrape data under the doctrine of fair use, but there is no hard and fast rule about fair use.

HTML basics

HTML has a hierarchical structure formed by **elements** which consist of tags and optional **attributes**, contents, and end tags.

A good reference on html tags is the MDN Web Docs created by the team behind mozilla(?).

Extracting data

Reading an HTML file using `read_html()`:

```
html <- read_html("http://rvest.org/")
html
```

```
## {html_document}
## <html lang="en">
## [1] <head>\n<meta http-equiv="Content-Type" content="text/html; charset=UTF-8 ...
## [2] <body onload="do_onload()">\n        <script type="text/javascript">\n    ...
```

Writing a simple html page using `minimal_html()`:

```
html <- minimal_html("
  <p>This is a paragraph</p>
  <ul>
    <li>This is a bulleted list</li>
  <>
")
html
```

```
## {html_document}
## <html>
## [1] <head>\n<meta http-equiv="Content-Type" content="text/html; charset=UTF-8 ...
## [2] <body>\n<p>This is a paragraph</p>\n  <ul>\n<li>This is a bulleted list</ ...
```

Find elements

CSS selector codes:
- `p` selects all `<p>` elements.
- `.title` selects all elements with class "title".
- `#title` selects all elements with the `id` attribute that equals "title". `Id` attributes must be unique within a document, so this will only ever select a single element.

Example:

```
html <- minimal_html("
  <h1>This is a heading</h1>
  <p id='first'>This is a parfagraph</p>
  <p class='important'>This is an important paragraph</p>
")
```

Using `html_elements()` to find all elements that match the selector:

```
html |> html_elements("p")
```

```
## {xml_nodeset (2)}
## [1] <p id="first">This is a parfagraph</p>
## [2] <p class="important">This is an important paragraph</p>
```

```
html |> html_elements(".important")
```

```
## {xml_nodeset (1)}
## [1] <p class="important">This is an important paragraph</p>
```

```
html |> html_elements("#first")
```

```
## {xml_nodeset (1)}
## [1] <p id="first">This is a parfagraph</p>
```

The other function `html_element()` always returns the same number of outputs as inputs:

```
html |> html_element("p")
```

```
## {html_node}
## <p id="first">
```

When you use a selector that doesn't match any elements, `html_elements()` returns a vector of length 0, while `html_element()` returns a missing value:

```
html |> html_elements("b")
```

```
## {xml_nodeset (0)}
```

```
html |> html_element("b")
```

```
## {xml_missing}
## <NA>
```

Nesting selections

Typically, `html_elements()` is used to identify elements that will become observations, and `html_element()` is used to find elements that will become variables. Example:

Creating the sample html document:

```
html <- minimal_html("
  <ul>
    <li><b>C-3PO</b> is a <i>droid</i> that weighs <span class='weight'>167 kg</span></li>
    <li><b>R4-P17</b> is a <i>droid</i></li>
    <li><b>R2-D2</b> is a <i>droid</i> that weighs <span class='weight'>96 kg</span></li>
    <li><b>Yoda</b> weighs <span class='weight'>66 kg</span></li>
  </ul>
  ")
```

Using `html_elements()` to make a vector where each element corresponds to a different character:

```
characters <- html |>  html_elements("li")
characters
```

```
## {xml_nodeset (4)}
## [1] <li>\n<b>C-3PO</b> is a <i>droid</i> that weighs <span class="weight">167 ...
## [2] <li>\n<b>R4-P17</b> is a <i>droid</i>\n</li>
## [3] <li>\n<b>R2-D2</b> is a <i>droid</i> that weighs <span class="weight">96  ...
## [4] <li>\n<b>Yoda</b> weighs <span class="weight">66 kg</span>\n</li>
```

Extracting the name of each character using `html_element()`:

```
characters |> html_element("b")
```

```
## {xml_nodeset (4)}
## [1] <b>C-3PO</b>
## [2] <b>R4-P17</b>
## [3] <b>R2-D2</b>
## [4] <b>Yoda</b>
```

Getting the weigh value for each element:

```
characters |> html_element(".weight")
```

```
## {xml_nodeset (4)}
## [1] <span class="weight">167 kg</span>
## [2] NA
## [3] <span class="weight">96 kg</span>
## [4] <span class="weight">66 kg</span>
```

We got an `NA` since there are only three weight elements that are children `characters`. Using `html_elements()`:

```
characters |> html_elements(".weight")
```

```
## {xml_nodeset (3)}
## [1] <span class="weight">167 kg</span>
## [2] <span class="weight">96 kg</span>
## [3] <span class="weight">66 kg</span>
```

Text and attributes

Extracting plain text contents of an html element using `html_text2()`:

```
characters |>
  html_element("b") |>
  html_text2()
```

```
## [1] "C-3PO"  "R4-P17" "R2-D2"  "Yoda"
```

```
characters |>
  html_element(".weight") |>
  html_text2()
```

```
## [1] "167 kg" NA       "96 kg"  "66 kg"
```

The **rvest** package automatically handles escape characters.

Extracting data from attributes using `html_attr()`:

```
html <- minimal_html("
  <p><a href='https://en.wikipedia.org/wiki/Cat'>cats</a></p>
  <p><a href='https://en.wikipedia.org/wiki/Dog'>dogs</a></p>
")

html |>
  html_elements("p") |>
  html_elements("a") |>
  html_attr("href")
```

```
## [1] "https://en.wikipedia.org/wiki/Cat" "https://en.wikipedia.org/wiki/Dog"
```

Tables

HTML tables are built up from four main elements:
- `<table>`: the table itself.
- `<tr>`: table row.
- `<th>`: table heading.
- `<td>`: table data.

Sample html with table:

```
html <- minimal_html("
  <table class='mytable'>
  <tr><th>x</th>   <th>y</th></tr>
  <tr><td>1.5</td> <td>2.7</td></tr>
  <tr><td>4.9</td> <td>1.3</td></tr>
  <tr><td>7.2</td> <td>8.1</td></tr>
  </table>
")
```

Extracting table data using `html_table()`:

```
html |>
  html_element(".mytable") |>
  html_table()
```

```
## # A tibble: 3 x 2
##       x     y
##   <dbl> <dbl>
## 1   1.5   2.7
## 2   4.9   1.3
## 3   7.2   8.1
```

The values are automatically converted to numbers.

Finding the right selectors

- Often, some experimenting is needed to find a selector that is both specific (doesn't select everything you don't care about) and sensitive (does select everything you care about).
- the tool SelectorGadget is useful here.
- The web toolkit for developers provided by Chrome is one of the best.
- The CSS dinner tutorial on CSS selectors is highly recommended.

Putting it all together

StarWars

Structure of the target web page:

```
<section>
  <h2 data-id="1">The Phantom Menace</h2>
  <p>Released: 1999-05-19</p>
  <p>Director: <span class="director">George Lucas</span></p>

  <div class="crawl">
    <p>...</p>
    <p>...</p>
    <p>...</p>
  </div>
</section>
```

Goal: turn the data on the web page into a 7-row data frame with variables `title`, `year`, `director`, and `intro`.

Reading the HTML and extracting all the `<section>` elements:

```
url <- "https://rvest.tidyverse.org/articles/starwars.html"
html <- read_html(url)

section <- html |> html_elements("section")
section
```

```
## {xml_nodeset (7)}
## [1] <section><h2 data-id="1">\nThe Phantom Menace\n</h2>\n<p>\nReleased: 1999 ...
## [2] <section><h2 data-id="2">\nAttack of the Clones\n</h2>\n<p>\nReleased: 20 ...
## [3] <section><h2 data-id="3">\nRevenge of the Sith\n</h2>\n<p>\nReleased: 200 ...
## [4] <section><h2 data-id="4">\nA New Hope\n</h2>\n<p>\nReleased: 1977-05-25\n ...
## [5] <section><h2 data-id="5">\nThe Empire Strikes Back\n</h2>\n<p>\nReleased: ...
## [6] <section><h2 data-id="6">\nReturn of the Jedi\n</h2>\n<p>\nReleased: 1983 ...
## [7] <section><h2 data-id="7">\nThe Force Awakens\n</h2>\n<p>\nReleased: 2015- ...
```

The code above retrieves seven elements matching the seven movies found on the web page, suggesting that using `section` as a selector is good.

Extracting the individual elements:

```
section |> html_element("h2") |> html_text2()
```

```
## [1] "The Phantom Menace"      "Attack of the Clones"
## [3] "Revenge of the Sith"     "A New Hope"
## [5] "The Empire Strikes Back" "Return of the Jedi"
## [7] "The Force Awakens"
```

```
section |> html_element(".director") |> html_text2()
```

```
## [1] "George Lucas"     "George Lucas"     "George Lucas"     "George Lucas"
## [5] "Irvin Kershner"   "Richard Marquand" "J. J. Abrams"
```

Once we are sure that the results are what we want, we can wrap them up into a tibble:

```
tibble(
  title = section |>
    html_element("h2") |>
    html_text2(),
  released = section |>
    html_element("p") |>
    html_text2() |>
    str_remove("Released: ") |>
    parse_date(),
  director = section |>
    html_element(".director") |>
    html_text2(),
  intro = section |>
    html_element(".crawl") |>
    html_text2()
)
```

```
## # A tibble: 7 x 4
##   title                  released   director         intro
##   <chr>                  <date>     <chr>            <chr>
## 1 The Phantom Menace     1999-05-19 George Lucas     "Turmoil has engulfed the~
## 2 Attack of the Clones   2002-05-16 George Lucas     "There is unrest in the G~
## 3 Revenge of the Sith    2005-05-19 George Lucas     "War! The Republic is cru~
## 4 A New Hope             1977-05-25 George Lucas     "It is a period of civil ~
## 5 The Empire Strikes Back 1980-05-17 Irvin Kershner   "It is a dark time for th~
## 6 Return of the Jedi     1983-05-25 Richard Marquand "Luke Skywalker has retur~
## 7 The Force Awakens      2015-12-11 J. J. Abrams     "Luke Skywalker has vanis~
```

IMDB top films

Using `html_table()` to read the IMDB web page:

```
url <- "https://web.archive.org/web/20220201012049/https://www.imdb.com/chart/top/"

html <- read_html(url)

table <- html |>
  html_element("table") |>
  html_table()
table
```

```
## # A tibble: 250 x 5
##    ``    `Rank & Title`                          `IMDb Rating` `Your Rating` ``
##    <lgl> <chr>                                          <dbl> <chr>         <lgl>
##  1 NA    "1.\n      The Shawshank Redemption\~           9.2 "12345678910~ NA
##  2 NA    "2.\n      The Godfather\n        (1~           9.1 "12345678910~ NA
##  3 NA    "3.\n      The Godfather: Part II\n ~           9   "12345678910~ NA
##  4 NA    "4.\n      The Dark Knight\n        ~           9   "12345678910~ NA
##  5 NA    "5.\n      12 Angry Men\n        (19~           8.9 "12345678910~ NA
##  6 NA    "6.\n      Schindler's List\n       ~           8.9 "12345678910~ NA
##  7 NA    "7.\n      The Lord of the Rings: Th~           8.9 "12345678910~ NA
##  8 NA    "8.\n      Pulp Fiction\n        (19~           8.8 "12345678910~ NA
##  9 NA    "9.\n      The Good, the Bad and the~           8.8 "12345678910~ NA
## 10 NA    "10.\n      The Lord of the Rings: T~           8.8 "12345678910~ NA
## # i 240 more rows
```

Processung further:

```
ratings <- table |>
  select(
    rank_title_year = `Rank & Title`,
    rating = `IMDb Rating`
  ) |>
  mutate(
    rank_title_year = str_replace_all(rank_title_year, "\n +", " ")
  ) |>
  separate_wider_regex(
    rank_title_year,
    patterns = c(
      rank = "\\d+", "\\. ",
      title = ".+", " +\\(",
      year = "\\d+", "\\)"
    )
  )
ratings
```

```
## # A tibble: 250 x 4
##    rank  title                                    year  rating
##    <chr> <chr>                                    <chr>  <dbl>
##  1 1     The Shawshank Redemption                 1994     9.2
##  2 2     The Godfather                            1972     9.1
```

```
##  3 3     The Godfather: Part II                                1974     9
##  4 4     The Dark Knight                                       2008     9
##  5 5     12 Angry Men                                          1957     8.9
##  6 6     Schindler's List                                      1993     8.9
##  7 7     The Lord of the Rings: The Return of the King         2003     8.9
##  8 8     Pulp Fiction                                          1994     8.8
##  9 9     The Good, the Bad and the Ugly                        1966     8.8
## 10 10    The Lord of the Rings: The Fellowship of the Ring 2001     8.8
## # i 240 more rows
```

"Spelunking" the source web page for additional info:

```
html |>
  html_elements("td strong") |>
  head() |>
  html_attr("title")
```

```
## [1] "9.2 based on 2,536,415 user ratings" "9.1 based on 1,745,675 user ratings"
## [3] "9.0 based on 1,211,032 user ratings" "9.0 based on 2,486,931 user ratings"
## [5] "8.9 based on 749,563 user ratings"   "8.9 based on 1,295,705 user ratings"
```

Combining this result with the previous result:

```
ratings |>
  mutate(
    rating_n = html |> html_elements("td strong") |> html_attr("title")
  ) |>
  separate_wider_regex(
    rating_n,
    patterns = c(
      "[0-9.]+ based on ",
      number = "[0-9,]+",
      " user ratings"
    )
  ) |>
  mutate(
    number = parse_number(number)
  )
```

```
## # A tibble: 250 x 5
##    rank  title                                                 year  rating  number
##    <chr> <chr>                                                 <chr>  <dbl>   <dbl>
##  1 1     The Shawshank Redemption                              1994     9.2 2536415
##  2 2     The Godfather                                         1972     9.1 1745675
##  3 3     The Godfather: Part II                                1974     9   1211032
##  4 4     The Dark Knight                                       2008     9   2486931
##  5 5     12 Angry Men                                          1957     8.9  749563
##  6 6     Schindler's List                                      1993     8.9 1295705
##  7 7     The Lord of the Rings: The Return of the King         2003     8.9 1749722
##  8 8     Pulp Fiction                                          1994     8.8 1952864
##  9 9     The Good, the Bad and the Ugly                        1966     8.8  732557
## 10 10    The Lord of the Rings: The Fellowship of the Ring 2001     8.8 1771245
## # i 240 more rows
```

Dynamic sites

At the time of writing, this area is still being worked on. The **chromote** package is used to give the user additional tools to interact with the target website.